

A Lightweight Digital Twin Framework for Cloud Security Posture and Configuration Drift Detection

Yashaswini C

T. John Institute of Technology, Bengaluru, Karnataka, India
IIIT Trichy Funded Research Project

Abstract—Cloud environments change continuously as identities, storage permissions, network exposure, encryption controls, and operational safeguards are modified by administrators, applications, and automated infrastructure tooling. A security weakness can therefore arise not only from a vulnerable component but from a configuration that silently drifts away from an intended secure state. This paper presents a lightweight Digital Twin-oriented framework for cloud security posture and configuration drift detection. The prototype maintains a software representation of four simulated cloud resources, compares their current security-relevant state against JSON-defined baselines, classifies deviations by severity, converts the resulting deviations into a posture score, and exposes the findings through a Flask web dashboard. In addition to isolated configuration-change experiments, the framework supports a cumulative chained-attack experiment in which multiple compromises are applied without resetting the simulated environment, and a complementary frequency detector that identifies repeated changes to the same parameter within a time window. In the executed experiments, isolated scenarios produced posture scores of 75, 60, 75, 75, and 92 for S3 public exposure, IAM administrative access, SSH exposure, disabled encryption, and disabled monitoring, respectively. The chained sequence reduced the posture score from 100 to 75, 35, 10, 0, and 0 as additional deviations accumulated. The frequency probe observed four changes to the SSH exposure parameter within one hour against a threshold of three and generated one high-severity frequency alert. The results demonstrate the functional feasibility of combining baseline-driven drift detection, posture scoring, alerting, cumulative attack simulation, and temporal change-frequency analysis in a compact research prototype. The current evaluation uses a simulated cloud environment; live AWS synchronization, large-scale benchmarking, and statistical performance evaluation are reserved for future work.

Index Terms—Cloud security posture management, digital twin, configuration drift, cloud misconfiguration, continuous monitoring, risk scoring, security dashboard.

I. INTRODUCTION

Cloud infrastructures are designed to be elastic, programmable, and rapidly reconfigurable. The same properties that make cloud computing operationally attractive also make security state difficult to keep stable. Identity permissions can be altered, storage can become externally accessible, network controls can broaden, encryption can be disabled, and monitoring controls can be changed without requiring a physical change to the infrastructure. NIST guidance on public-cloud security emphasizes that organizations must account for the security and privacy implications of outsourcing infrastructure and services to cloud environments [1]. Continuous monitoring guidance further emphasizes maintaining visibility into assets, security controls, threats, vulnerabilities, and the effectiveness

of deployed controls over time [2]. Industry evidence likewise identifies cloud misconfiguration and visibility as practical CSPM challenges [4].

Traditional configuration checking usually answers a narrow question: whether a resource currently violates one or more expected security rules. That capability is necessary, but a cloud posture is temporal rather than static. A resource can move from a secure state to an insecure state, return to secure, and change repeatedly during a short interval. A detector that only compares the latest state with a baseline can identify the current deviation but cannot, by itself, express how a sequence of changes evolved or whether repeated toggling is itself suspicious. Cloud security automation research has therefore emphasized continuous validation of security controls rather than one-time manual inspection [3].

Digital Twin concepts provide a useful abstraction for this temporal problem because they maintain a digital representation of an operational counterpart and can support monitoring, simulation, and analysis. Recent cybersecurity literature has explored digital twins as enablers for threat modeling, detection, and security operations [5]–[7]. Other recent work has developed broader cybersecurity digital-twin architectures for multi-domain and multi-ownership environments [10]. These studies motivate the use of a persistent digital representation, while also highlighting the need for practical and experimentally grounded implementations.

This paper presents a lightweight framework, denoted DT-CSPM, that applies the Digital Twin idea to security posture and configuration drift in a controlled cloud-like environment. The contribution is intentionally bounded. The prototype does not claim to be a production CSPM platform, nor does it presently connect to a live cloud account. Instead, it provides a small and transparent research environment in which configuration state, baselines, drift detection, risk scoring, alert generation, attack composition, and temporal change frequency can be studied together.

The main contributions of this work are threefold. First, the framework combines a Digital Twin-style state representation with explicit security baselines and deterministic drift analysis. Second, it evaluates cumulative degradation through a chained attack sequence rather than testing all attacks independently. Third, it augments baseline comparison with a change-frequency detector, allowing repeated parameter changes to be analyzed as temporal behavior. The study also documents the implementation and its measured outputs so that the reported claims remain directly traceable to the experiment.

II. RELATED WORK

A. Cloud Security Posture and Misconfiguration

CSPM research focuses on continuously assessing cloud configurations against security expectations, policies, or compliance requirements. NIST's Cloud Security Automation Framework describes automated cloud configuration and continuous monitoring of security controls as a means of reducing dependence on manual approaches [3]. NIST SP 800-137 similarly frames continuous monitoring as an ongoing activity that supports risk-based decisions when control effectiveness changes [2]. A broader NIST cloud-security publication establishes the need to treat public-cloud security as a risk-management concern involving outsourced infrastructure and data [1].

Practical industry assessments also emphasize the prevalence and operational difficulty of misconfiguration. A Cloud Security Alliance survey reported that organizations identify security visibility and monitoring as major challenges related to cloud misconfiguration management [4]. Recent evaluations of CSPM tools further show that real-world detection performance can vary by tool, cloud platform, severity, and misconfiguration type, motivating careful benchmarking rather than assuming complete coverage [11].

The present work does not attempt to compete with mature CSPM products in scope. Instead, it isolates a narrower research question: whether a compact Digital Twin representation can make security-state changes easier to model, inspect, and experiment with while preserving a deterministic baseline-driven detector.

B. Digital Twins for Cybersecurity

Digital Twins have moved beyond manufacturing into cybersecurity applications. Recent reviews survey their use in security monitoring, threat modeling, intrusion detection, simulation, and security operations [5], [7]. Erceylan *et al.* investigated digital-twin-supported threat modeling for cyber-physical systems and emphasized simulation, continuous assessment, and non-disruptive analysis [6]. A recent framework for security-enhancing Digital Twins similarly treats the twin as a complement to existing security mechanisms for incident-response activities [8].

A January 2026 paper by Repetto proposes a cybersecurity Digital Twin architecture for cooperative security across multi-ownership digital service chains and explicitly considers multi-step attacks [10]. Baiardi and Sammartino present a simulation-oriented security twin for real-time risk assessment, emphasizing a dynamic security representation that can be used without disrupting live systems [9]. These works establish the relevance of a stateful digital representation, but they differ in application domain, architecture, and evaluation scope.

C. Research Gap and Positioning

The literature indicates that Digital Twins are increasingly used for cybersecurity, but many studies focus on industrial control systems, cyber-physical systems, broad security operations, threat modeling, or predictive analytics [5]–[7]. At the same

TABLE I: Representative related work and positioning of the present study.

Work	Main focus	Relation to this study
Dempsey <i>et al.</i> [2] Tunc <i>et al.</i> [3]	Continuous monitoring Cloud security automation	Motivates ongoing visibility of control state. Supports automated configuration validation.
Qureshi <i>et al.</i> [5]	DT cybersecurity survey	Establishes the broader DT-security landscape.
Erceylan <i>et al.</i> [6]	DT-assisted threat modeling	Demonstrates simulation and continuous analysis.
Empl <i>et al.</i> [7]	Security-enhancing DTs	Provides cybersecurity DT concepts and use cases.
Baiardi & Sammartino [9] Moss [11]	Security twins and risk assessment CSPM tool evaluation	Reinforces stateful, simulation-oriented security twins. Highlights variability in cloud misconfiguration detection.

time, CSPM work tends to focus on configuration assessment and policy compliance rather than on maintaining a research-oriented digital state representation for controlled attack-state experiments. The present framework therefore explores the intersection in a deliberately lightweight form: a Digital Twin-style representation centered on cloud security configuration state, deterministic drift comparison, cumulative attack-state evolution, and temporal change frequency.

The distinction is important. The paper does not claim that baseline comparison, risk scoring, alerting, or change logging are individually novel techniques. The research contribution is the integrated experimental framework and the evidence generated by applying that framework to isolated, cumulative, and repeated-change scenarios in one controlled environment. This positioning also makes the limitations explicit: the prototype is not yet a live bidirectional cloud twin, and its evaluation is functional rather than a large statistical benchmark.

Table I summarizes the role of representative prior work in positioning the study.

III. PROBLEM FORMULATION

Let a cloud resource r be represented by a finite set of security parameters P_r . For each parameter $p \in P_r$, the framework stores an expected baseline value $B(r, p)$ and a current simulated value $C(r, p)$. A configuration drift exists when

$$D(r, p) = \begin{cases} 1, & C(r, p) \neq B(r, p), \\ 0, & C(r, p) = B(r, p). \end{cases} \quad (1)$$

The framework assigns a severity w_s to each detected drift based on the parameter being violated. The implemented weights are 25 for CRITICAL, 15 for HIGH, 8 for MEDIUM, and 3 for LOW. For a set of detected drifts $\mathcal{D}$, the posture score is

$$S = \max \left(0, 100 - \sum_{d \in \mathcal{D}} w(d) \right). \quad (2)$$

The score is mapped to four operational levels by the implementation: SECURE for scores at least 80, MODERATE

TABLE II: Severity weights and operational risk labels used by the prototype.

Severity	Penalty	Score interpretation
CRITICAL	25	Below 40: critical risk
HIGH	15	40–59: high risk
MEDIUM	8	60–79: moderate
LOW	3	80–100: secure

for 60–79, HIGH RISK for 40–59, and CRITICAL RISK for values below 40. This score is intentionally transparent rather than probabilistic. It should therefore be interpreted as an application-specific posture indicator, not as a calibrated probability of compromise.

The framework also records state changes over time. For a resource-parameter pair (r, p) , let $T(r, p)$ be the list of change timestamps. A frequency alert is raised when the number of timestamps in a one-hour sliding window reaches the threshold of three:

$$F(r, p, t) = |\{\tau \in T(r, p) : t - \tau \leq 3600\}| \geq 3. \quad (3)$$

This second detector is deliberately different from drift detection. Baseline comparison identifies the current mismatch, whereas frequency analysis identifies unusually repeated state transitions.

IV. THREAT MODEL AND OPERATIONAL ASSUMPTIONS

The prototype assumes that configuration state can change without requiring a full application restart and that security-relevant configuration can be represented as structured attributes. The adversarial actions are modeled as controlled state mutations rather than exploits against a live provider. Thus, S3 public exposure represents a transition from a private to a public access setting; IAM escalation represents the assignment of administrative privileges and loss of least privilege; SSH exposure represents broad network access; encryption disabled represents loss of a data-protection control; and monitoring disabled represents a reduction in observability. These states correspond to security controls that cloud platforms commonly expose. For example, AWS documentation recommends restricting public access to S3 buckets and using access logging and CloudTrail for auditing, providing real-world motivation for the S3 controls represented in the prototype [12]. MITRE ATT&CK also documents cloud-account privilege and role misuse as a security concern, including situations in which over-privileged cloud accounts can be used to pivot or access additional resources [13].

The threat model is intentionally narrow. It focuses on post-change security-state observation and repeated configuration changes rather than exploitation of software vulnerabilities, credential theft, network packet inspection, or malware detection. The simulator is therefore a controlled experimental mechanism for evaluating state analysis. It should not be interpreted as an end-to-end attack platform.

A. Operational Assumptions

The framework assumes that baseline definitions are trusted and sufficiently specific for the monitored resource type. A missing or incomplete baseline can cause a control to be

TABLE III: Baseline controls represented by the simulated environment.

Resource	Baseline expectations
S3	Public access disabled; encryption, versioning, logging, and SSL enforcement enabled.
IAM	No administrative access; MFA enabled; access-key age set to 90 days; unused credentials removed; least privilege enabled.
Security group	SSH/RDP not open to all; unrestricted traffic and unused ports disabled; outbound traffic restricted.
VM	No public IP exposure; OS patched; monitoring enabled; encryption at rest and backups enabled.

unassessed rather than incorrectly labeled secure. The current implementation reads expected values from four JSON files and compares only the defined parameters. The model also assumes that configuration changes are captured when they occur through the Digital Twin update interface. This is appropriate for the present simulated environment, but a live deployment would require an ingestion path from provider APIs, configuration services, event buses, or audit logs.

B. Why a State-Centric Model?

The state-centric design makes the experimental logic inspectable. Instead of reducing each cloud resource immediately to a single alarm, the twin retains the resource identity, current parameter values, historical changes, and timestamps. The detector can then regenerate the current drift inventory from state. This distinction is useful during cumulative experiments because the sequence of changes remains visible even when the posture score reaches its lower bound. It also provides a natural location for future temporal analytics without changing the concept of the baseline comparison layer.

V. RESEARCH QUESTIONS AND DESIGN RATIONALE

The study is organized around three research questions (RQs). RQ1 asks whether a baseline-driven Digital Twin representation can identify controlled security-configuration deviations across heterogeneous simulated cloud resources. RQ2 asks whether the same representation can preserve and expose cumulative security degradation when multiple attack stages are applied without resetting the environment. RQ3 asks whether a simple temporal frequency rule can provide complementary evidence when the same security parameter is changed repeatedly. These questions are intentionally operational. They are designed to be answered by the implemented prototype and its recorded experiment outputs rather than by claims about production-scale detection performance.

The design follows four principles. First, the baseline must be explicit so that every detected deviation has a traceable expected value. Second, the Digital Twin must preserve current state so that a later experiment can build on earlier changes. Third, scoring should be transparent enough that a reviewer can reproduce every posture value from the drift list. Fourth, temporal change information should be retained rather than discarded after the latest configuration is computed.

The design also separates the simulation layer from the analysis layer. Attack scenarios are controlled state transitions and are not themselves detectors. This separation makes it possible to apply the same drift detector to all scenarios. It also means that future live-cloud ingestion can be evaluated as a

replacement for the simulator without changing the conceptual analysis chain.

VI. PROPOSED FRAMEWORK

A. System Overview

The implemented framework contains six functional blocks: baseline storage, Digital Twin core, drift detector, risk scorer, alert system, and dashboard. A seventh logical function, change-frequency detection, consumes the change history maintained by the twin. The design is lightweight and uses JSON baselines, Python modules, and a Flask interface.

B. Baseline Definition Engine

Four baseline files define expected security controls for an S3 bucket, IAM role, security group, and virtual machine. The baselines are intentionally explicit key-value structures. Examples include disallowing public S3 access, requiring encryption, requiring least privilege and MFA, disallowing SSH access from all addresses, and requiring VM monitoring.

C. Digital Twin Core

The Digital Twin core stores each resource as a dictionary containing its type, current configuration, timestamp, and active status. The final implementation also maintains a change log and a general history of configuration transitions. An important implementation refinement was to merge incoming updates with the existing configuration rather than replace the entire configuration. This makes cumulative attack experiments faithful to the attack sequence because a later scenario changes only the parameters it intends to modify while preserving earlier compromises.

The implementation should be described as a lightweight Digital Twin-style representation rather than as a production-scale cloud twin. It reproduces the security-relevant state of simulated resources and supports repeated analysis, but it does not presently synchronize bidirectionally with a live cloud provider.

D. Drift Detection

The drift detector iterates through every monitored resource, obtains its resource type, retrieves the corresponding baseline, and compares each baseline parameter with the current value. When the values differ, a drift record is generated containing the resource identifier, parameter, expected value, observed value, timestamp, severity, and a remediation message. Severity is deterministic and parameter-based. Public exposure, administrative access, encryption failures, SSH exposure, and several other security-control violations are mapped to CRITICAL; identity and control-strength parameters such as least privilege or MFA are mapped to HIGH; logging, monitoring, backups, and patch status are mapped to MEDIUM; all remaining deviations default to LOW.

E. Risk Scoring

The risk scorer converts the drift set into a posture score using the fixed severity weights described in Section III. The score is not intended to reproduce a recognized industry risk model; it is a transparent experimental metric designed to expose the effect of configuration changes and cumulative violations within the prototype. This distinction is important when interpreting the numerical results.

F. Alert System

The alert system converts CRITICAL and HIGH drift records into open alerts. During implementation, an alert-duplication problem was observed because the dashboard performs periodic scans. The final implementation suppresses a new alert when an open alert already exists for the same resource and parameter. Repeated scans may therefore still detect the same drift, but they do not create a new alert for an already open condition. This separates event observation from alert generation and keeps the dashboard usable during continuous polling.

G. Change-Frequency Detection

The frequency detector operates on the Digital Twin's change log. Each actual parameter transition stores a timestamp under a resource-parameter key. The detector evaluates changes occurring within the most recent one-hour window and raises a HIGH-severity frequency alert when at least three changes are observed. The feature is intended as a complementary temporal signal rather than a replacement for baseline comparison. Its motivation is consistent with continuous-monitoring principles in NIST guidance, which emphasize observation at frequencies appropriate to risk-based decisions [2].

H. Dashboard

A Flask web application exposes the security posture through a browser-based interface. The dashboard displays the current posture score, risk level, resource count, severity summary, simulated attack controls, reset control, and detailed drift alerts. The page polls the scan endpoint periodically, allowing the interface to reflect configuration changes without manual reloads. Figure 5 shows the dashboard during an S3 public-access test, while Figure 6 shows the restored baseline state.

VII. DETECTION AND PROCESSING WORKFLOW

The full processing cycle can be expressed as a sequence of deterministic operations. First, the environment is initialized with baseline configurations and resource state. Second, a scenario or external update changes one or more parameters. Third, the drift detector compares the new state with the appropriate baseline. Fourth, the risk scorer applies the severity weights. Fifth, the alert layer generates a high-priority alert only for CRITICAL and HIGH deviations and suppresses duplicates for an already open resource-parameter condition. In parallel, each actual parameter change is appended to the Digital Twin change log. The frequency detector later evaluates this log independently.

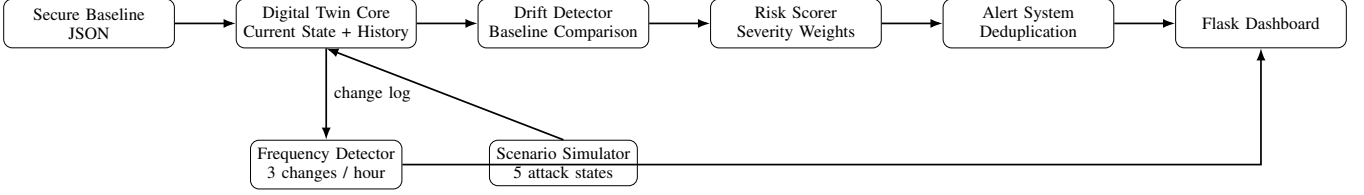


Fig. 1: Architecture of the implemented framework. The Digital Twin core holds the current simulated resource state and historical changes; deterministic baseline comparison and frequency analysis provide complementary detection paths.

- 1) Initialize baselines and resource state.
- 2) Apply an isolated or cumulative configuration change.
- 3) Compare current values against baseline values.
- 4) Construct drift records with severity and remediation data.
- 5) Calculate the posture score and severity summary.
- 6) Generate or suppress alerts according to the active-alert key.
- 7) Record parameter-change timestamps for temporal analysis.

The workflow deliberately separates three concepts: configuration state, detection evidence, and operator notification. A drift can be detected repeatedly during dashboard polling while only one corresponding active alert is retained. This separation was important in the implementation because the dashboard refresh cycle previously created duplicate alert records.

A. Configuration Update Semantics

A critical implementation detail is the distinction between replacing a configuration and applying a partial update. The final Digital Twin core copies the existing resource configuration and then updates only the keys supplied by a scenario. This allows an attack chain to retain an earlier violation while introducing a new one. The same mechanism is used by the reset operation, which supplies the complete secure configuration for each resource and therefore restores every monitored attribute to its baseline value.

B. Alert Identity and Deduplication

An alert is uniquely treated as an active condition by the pair (resource identifier, parameter). When a dashboard refresh encounters the same still-open drift, the detector records the drift but the alert layer suppresses a second notification. This is an operational deduplication rule rather than a drift-detection rule. Its effect is visible in the observed dashboard behavior: repeated API scan requests continue to report the S3 public-access drift, while the alert count remains at one.

VIII. IMPLEMENTATION

The prototype is implemented in Python. Baseline information is stored in four JSON documents, attack scenarios are implemented as deterministic functions, and Flask provides the web layer. The final project consists of a Digital Twin core, drift detection, risk scoring, alerting, attack simulations, dashboard routes, HTML templates, and CSS styling.

The four simulated resources are: `s3-prod-data` (S3 bucket), `iam-developer-role` (IAM role), `sg-web-server` (security group), and `vm-app-server`

TABLE IV: Simulated resources and representative security controls.

Resource	Representative controls
S3 bucket	Public access, encryption, versioning, logging, SSL enforcement
IAM role	Administrative access, MFA, key age, credential cleanup, least privilege
Security group	SSH exposure, RDP exposure, unrestricted traffic, unused ports, outbound restriction
VM instance	Public IP exposure, OS patch state, monitoring, at-rest encryption, backups

TABLE V: Implemented attack scenarios.

ID	Scenario	Main altered control
S1	S3 public exposure	<code>public_access=True</code>
S2	IAM administrative access	<code>admin_access=True</code> , <code>least_privilege=False</code>
S3	SSH open to all	<code>ssh_open_to_all=True</code>
S4	Encryption disabled	<code>encryption=False</code>
S5	Monitoring disabled	<code>monitoring_enabled=False</code>

(VM instance). Their configuration attributes correspond to common cloud security controls. The scenario simulator modifies these attributes to create controlled deviations from baseline.

IX. EXPERIMENTAL SETUP

A. Evaluation Objectives

The experiments were designed to answer three practical questions. First, can the framework consistently identify individual configuration deviations relative to secure baselines? Second, can the Digital Twin retain and expose accumulated deviations when multiple attack stages are applied without resetting the environment? Third, can the separate frequency detector recognize repeated configuration changes that may not be distinguishable by examining only the latest state?

B. Isolated Scenario Procedure

For the isolated evaluation, the system starts from a secure baseline. One scenario is applied and a scan is performed. The environment is then reset to the secure baseline before the next scenario. This makes the scenarios independent, so each score reflects only the selected attack state. The final automatic run also verifies that the environment returns to a posture score of 100 with no detected drifts after the final reset.

C. Chained Attack Procedure

The chained experiment starts from the secure baseline and does not reset between attack stages. The sequence is: SSH

TABLE VI: Measured isolated-scenario results.

Scenario	Score	Drifts	C	H	M	Alerts
Baseline	100	0	0	0	0	0
S1 S3 public	75	1	1	0	0	1
S2 IAM admin	60	2	1	1	0	2
S3 SSH open	75	1	1	0	0	1
S4 Encryption off	75	1	1	0	0	1
S5 Monitoring off	92	1	0	0	1	0
Restored	100	0	0	0	0	0

exposure, IAM privilege escalation, encryption disabled, S3 public exposure, and monitoring disabled. After each stage, the system scans the complete Digital Twin state. Because the final update mechanism merges parameter changes, previously introduced deviations persist into subsequent stages. This experiment therefore measures cumulative posture degradation rather than isolated effect.

D. Frequency Probe Procedure

The frequency probe repeatedly toggles the SSH exposure control. Four actual state transitions are produced within a short execution window, well inside the one-hour analysis horizon. The frequency detector then counts recent changes for the resource-parameter pair and compares the count with a threshold of three. The experiment records the number of observed changes and whether a HIGH frequency alert is produced.

E. Scope of Metrics

The current prototype measures posture score, number of drift records, severity distribution, alert counts, cumulative drift counts in the chained experiment, and frequency-probe outcomes. The present experiment does not provide sufficient repeated observations to estimate accuracy, precision, recall, F1-score, false-positive rate, false-negative rate, latency distributions, or statistical confidence intervals. Those metrics are therefore not presented as current results.

X. RESULTS AND DISCUSSION

A. Isolated Attack Results

Table VI reports the measured outputs from the final automatic five-scenario execution. The secure baseline produced a score of 100 with no detected drift. S1, S3, and S4 each produced one critical drift and a score of 75. S2 produced two deviations, one critical and one high, and reduced the score to 60. S5 produced one medium drift and a score of 92. After restoration, the score returned to 100.

The results reflect the deterministic penalty model. A single critical violation subtracts 25 points, leading to the 75-point scores observed in S1, S3, and S4. S2 produces a 25-point critical penalty and a 15-point high-severity penalty for a total score of 60. S5 produces an 8-point medium penalty, giving 92. These relationships are directly implied by the implemented scoring rule rather than by an externally calibrated risk standard.

The absence of alerts for S5 is also consistent with the alert policy. The drift is classified as MEDIUM, while the alert generator creates alerts only for CRITICAL and HIGH

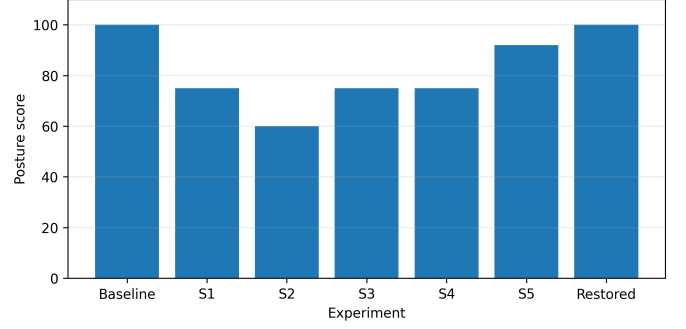


Fig. 2: Posture scores measured for the isolated scenarios and restored baseline.

TABLE VII: Measured cumulative chained-attack progression.

Stage	Score	Drifts	C	H	M	Alerts
Baseline	100	0	0	0	0	0
SSH	75	1	1	0	0	1
+ IAM	35	3	2	1	0	2
+ Encryption	10	4	3	1	0	1
+ S3 public	0	5	4	1	0	1
+ Monitoring	0	6	4	1	1	0

findings. The result should therefore not be interpreted as a failure to detect S5; the system detects the drift but deliberately does not generate a high-priority alert for that severity.

B. Chained Attack Results

The chained attack produced a clear cumulative trajectory. The score fell from 100 at baseline to 75 after SSH exposure, to 35 after the IAM escalation stage, to 10 after encryption was disabled, and then to 0 when S3 public access was added. The monitoring stage kept the score at 0 while increasing the cumulative drift count from five to six.

The cumulative drift count increased monotonically from 0 to 6. This provides a useful functional demonstration that later attack stages retain earlier violations when the Digital Twin state is updated incrementally. The score reaches zero before the final stage, which means the score is saturated by the fixed penalty model. Consequently, the final two stages cannot be distinguished through the score alone even though the drift inventory increases from five to six. This is an important observation: the posture score communicates a coarse state, whereas the drift inventory retains finer-grained information.

The alert counts also illustrate the distinction between detection and alert deduplication. At a later chain stage, previously open high-severity alerts are not recreated for every repeated scan. The additional alert appears only when a newly introduced critical or high deviation is encountered. This behavior prevents the dashboard from treating every polling cycle as a new incident while still retaining the current drift state.

C. Frequency Detection Results

The frequency experiment generated four actual SSH configuration changes within the one-hour evaluation window. Since the threshold was three, the detector emitted one HIGH frequency alert. The result confirms that temporal repetition can

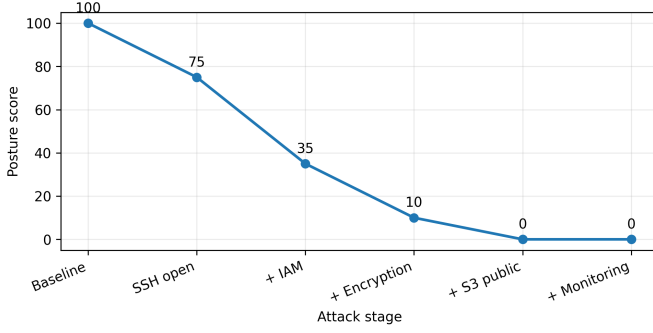


Fig. 3: Posture-score progression during the cumulative chained attack.

TABLE VIII: Frequency-detection experiment.

Resource	Parameter	Changes	Window	Thre
sg-web-server	ssh_open_to_all	4	1 h	

produce a distinct detection signal even when the configuration is returned to its baseline state between toggles.

The frequency detector should be interpreted carefully. The experiment demonstrates the operation of a threshold-based temporal rule; it does not establish that four changes are malicious in a production environment. A legitimate administrator or automation pipeline might produce the same pattern. Therefore, the current result supports the feasibility of temporal change monitoring, while contextual baselines, asset criticality, user identity, change authorization, and maintenance windows would be required for a production-grade behavioral detector.

D. Dashboard Evidence

The dashboard provides a visual representation of the experimental state. In the S3 public-access experiment, the interface displayed a posture score of 75, a CRITICAL S3 drift, and one open alert. The screenshot also showed other monitored resources in a secure state. This is consistent with the terminal result, where one critical drift was detected for `public_access`. The secure-state screenshot shows a score of 100, zero critical/high/medium drift counts, and a message indicating that no drifts were detected.

The dashboard is therefore useful as an observability layer rather than as an independent detection mechanism. Detection logic remains in the Python modules, while the browser interface communicates the resulting state to a human operator. This separation keeps the experimental logic independent from presentation.

E. Experiment-Level Reproducibility

The evaluation is reproducible because all attack states are represented by deterministic configuration updates. The isolated experiment resets the complete resource state after each scenario. The chained experiment deliberately avoids reset calls between attack stages, while the frequency probe alternates the SSH control between open and secure states. The resulting logs expose the exact score and drift count at each scan. This makes the study suitable for controlled functional validation even though it is not a production-scale benchmark.

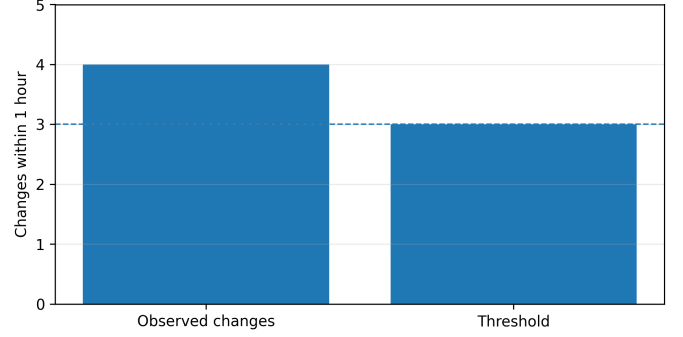


Fig. 4: Observed SSH-configuration changes versus the frequency-alert threshold.



Fig. 5: Dashboard during S3 public-access exposure: posture score 75 with one critical drift and one open alert.

The three experiments expose complementary aspects of cloud security posture. Isolated tests demonstrate sensitivity to individual security-control violations. The chained test demonstrates persistence and accumulation of security-state deviations. The frequency experiment demonstrates that the history of changes contains information not present in the latest state alone. Together, these observations support the design decision to maintain both current configuration and change history inside the Digital Twin core. Isolated tests demonstrate sensitivity to individual security-control violations. The chained test demonstrates persistence and accumulation of security-state deviations. The frequency experiment demonstrates that the history of changes contains information not present in the latest state alone. Together, these observations support the design decision to maintain both current configuration and change history inside the Digital Twin core.

The findings should nevertheless be read within the bounds of the implementation. The score function is deterministic and bounded at zero; it therefore compresses multiple severe deviations into the same terminal score. The attack scenarios are controlled mutations of a simulated environment rather than organic cloud events. The frequency detector uses a fixed threshold, not a learned or asset-specific baseline. These constraints make the prototype transparent and reproducible, but they also limit its direct transfer to production environments.

XI. LIMITATIONS

Several limitations shape the interpretation of this study. First, the current cloud environment is simulated. The four resources and their security states are represented inside the Digital Twin

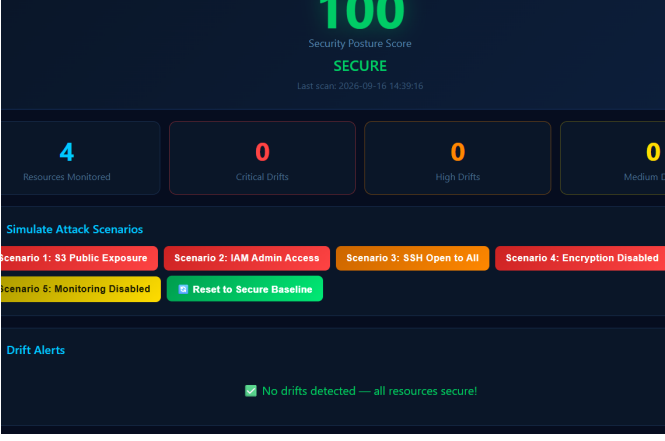


Fig. 6: Dashboard after restoration to the secure baseline: posture score 100 and no detected drifts.

TABLE IX: Purpose of the three experimental modes.

Mode	What changes	Primary question
Isolated	One scenario, then reset	Can an individual deviation be detected and scored?
Chained	Deviations accumulate without reset	How does posture evolve as compromises accumulate?
Frequency	Same parameter changes repeatedly	Can temporal repetition be flagged separately from current-state drift?

rather than retrieved continuously from an actual AWS account. The framework therefore demonstrates the detection logic and experimental workflow, not live-cloud synchronization.

Second, the evaluated resource set is deliberately small. Four resources and five scenarios are sufficient for a controlled prototype, but they do not represent the diversity of production-scale cloud services, organizational policies, account structures, dependencies, or multi-cloud environments.

Third, the evaluation uses deterministic scenario execution and one frequency-probe sequence. This is adequate for functional verification but insufficient for statistical characterization. In particular, it is not possible to derive a defensible detection-accuracy percentage, false-positive rate, or latency distribution from the current evidence alone.

Fourth, the posture score is a research metric defined by fixed severity weights. It provides useful relative changes within the prototype but is not a standardized industry risk score. Its thresholds and weights would need empirical validation or policy calibration before they were used to support operational decisions.

Finally, the present framework is best described as a lightweight Digital Twin-style security representation. It retains state and history and enables simulation, but it does not yet provide the full bidirectional synchronization and operational integration associated with mature Digital Twin deployments.

XII. FUTURE WORK

The most direct next step is live cloud integration. The architecture can be extended with a provider adapter that

retrieves security-relevant configuration from AWS through supported APIs, including boto3-based connectors, and maps those observations into the same resource model currently used by the simulator. In such a design, the existing drift detector and risk scorer could remain conceptually unchanged while the simulator is replaced or supplemented by live configuration ingestion.

A second direction is multi-cloud support. Provider-specific adapters could translate AWS, Azure, and other cloud configuration objects into a normalized security-state representation. This would make the Digital Twin layer less dependent on provider-specific syntax while preserving provider-specific checks.

A third direction is richer temporal reasoning. The current frequency detector is deliberately simple. Future work can incorporate authorized-change context, user identity, maintenance schedules, asset criticality, and sequences of related parameter changes. Temporal correlation could then distinguish routine automation from suspicious repeated probing with greater context.

A fourth direction is quantitative evaluation at larger scale. Repeated trials across a larger resource inventory could support the calculation of detection latency, false-positive and false-negative rates, precision, recall, F1-score, and scalability measures. Such a study should use a clearly defined ground-truth protocol rather than infer performance from functional demonstrations.

A live-cloud architecture can be developed as a provider-adapter layer rather than by rewriting the core detectors. In such a design, AWS configuration APIs and event sources would populate the current-state model, while the existing baseline comparison would consume normalized resource objects. A write-back or remediation channel could remain optional and separately controlled, preventing the experimental detector from automatically modifying production resources.

Finally, future research can investigate whether the Digital Twin can support pre-change security simulation. A proposed configuration could be applied to a replica first, allowing the system to estimate how the posture score and drift inventory would change before the configuration is introduced into a live environment. This would move the framework from observation toward risk-aware validation while preserving the current emphasis on transparent state representation.

XIII. CONCLUSION

This paper presented a lightweight Digital Twin-oriented framework for cloud security posture and configuration drift detection. The prototype models four simulated cloud resources against JSON-defined secure baselines, detects configuration deviations, assigns deterministic severity values, computes a posture score, generates alerts, and exposes the state through a Flask dashboard. The implementation was extended with duplicate-alert suppression, cumulative chained-attack execution, and a change-frequency detector.

The measured experiments show that the framework successfully identifies the five implemented security scenarios, returns the environment to a score of 100 after restoration, tracks cumulative drift during a chained sequence, and detects repeated

SSH configuration changes through a temporal threshold rule. The chained experiment produced the score progression 100, 75, 35, 10, 0, and 0, while the frequency probe observed four changes against a threshold of three and generated one HIGH frequency alert.

The work is intentionally presented as a controlled research prototype rather than a production CSPM deployment. Its value is the integration of state representation, deterministic baseline comparison, cumulative scenario simulation, alerting, and temporal change analysis in a compact and inspectable framework. Live cloud integration, broader empirical validation, and statistically grounded performance measurement remain important future steps.

DATA AND CODE AVAILABILITY

The study uses researcher-developed simulated resource configurations and controlled attack scenarios. The implementation, baseline definitions, and experiment logs form the basis of the reported results. A public repository was not assumed in this manuscript; code and supporting artifacts may be made available by the researcher according to project and institutional requirements.

ACKNOWLEDGMENT

The author acknowledges the support of the IIIT Trichy funded research project under which this work was developed.

CONFLICT OF INTEREST

The author declares no conflict of interest associated with this study.

APPENDIX A: ALGORITHMIC SPECIFICATION AND REPRODUCIBILITY NOTES

A.1 Drift Detection Procedure

The drift-detection procedure used in the prototype can be summarized by the following pseudocode.

```
for each resource r:
    baseline = baseline_for_type(r.type)
    for each parameter p in baseline:
        if current[r][p] != baseline[p]:
            create drift record
            assign severity(p)
            attach remediation(p)
return drift list
```

The algorithm is intentionally deterministic. The same resource state and baseline produce the same drift inventory and posture score, independent of dashboard presentation.

A.2 Alert Generation Procedure

For each CRITICAL or HIGH drift, the alert system computes an active-condition key consisting of the resource identifier and parameter name. If an open alert with that key already exists, no additional alert record is created. Otherwise, a new alert is appended. This design was introduced to separate polling frequency from incident multiplicity.

A.3 Frequency Detection Procedure

The frequency detector reads the timestamp list associated with each resource-parameter pair. At analysis time t , it retains timestamps τ satisfying $t - \tau \leq 3600$ seconds. A HIGH frequency alert is emitted when at least three timestamps remain. The experiment intentionally uses four changes so that the observed count exceeds the configured threshold without requiring an artificially long waiting period.

A.4 Reproducibility Checklist

A clean execution begins with four baseline resource states and no prior runtime history. The isolated experiment selects one scenario at a time and resets every resource before the next case. The chained experiment deliberately omits reset operations between attack stages. The frequency experiment repeatedly toggles the SSH exposure control. At the end of the isolated test, a final scan verifies a score of 100 and zero drifts. The experiment logs used for the present manuscript were captured from the Windows-based development environment on September 16, 2026.

The implementation is small enough that the principal score transitions can be independently reconstructed. For example, one CRITICAL drift incurs a 25-point penalty, two deviations consisting of one CRITICAL and one HIGH incur 40 points of total penalty, and one MEDIUM drift incurs an 8-point penalty. The observed values 75, 60, and 92 therefore follow directly from the implemented scoring rule.

A.5 Interpretation Boundary

The reproducibility of a deterministic prototype should not be confused with external validity. Reproducing the same simulated scenario establishes that the software behaves consistently under the specified state transitions. It does not establish how the framework would perform against uncontrolled cloud workloads, unknown attack patterns, provider-specific API semantics, or large-scale operational noise. These questions remain part of the future evaluation plan.

REFERENCES

- [1] W. Jansen and T. Grance, "Guidelines on Security and Privacy in Public Cloud Computing," NIST Special Publication 800-144, National Institute of Standards and Technology, 2011, doi: 10.6028/NIST.SP.800-144.
- [2] K. L. Dempsey, N. S. Chawla, L. A. Johnson, R. Johnston, A. C. Jones, A. Orebaugh, M. Scholl, and K. Stine, "Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations," NIST Special Publication 800-137, National Institute of Standards and Technology, 2011, doi: 10.6028/NIST.SP.800-137.
- [3] C. Tunc, S. Hariri, M. Merzouki, C. Mahmoudi, F. de Vaulx, J. Chbili, R. Bohn, and A. Battou, "Cloud Security Automation Framework," in *Proc. IEEE Workshop on Automation of Cloud Configuration and Operations*, 2017, doi: 10.1109/FAS-W.2017.164.
- [4] Cloud Security Alliance, "State of Cloud Security Risk, Compliance, and Misconfigurations," survey report, 2021.
- [5] A. R. Qureshi, A. Asensio, M. Imran, J. Garcia, and X. Masip-Bruin, "A survey on security enhancing Digital Twins: Models, applications and tools," *Computer Communications*, vol. 238, art. no. 108158, 2025, doi: 10.1016/j.comcom.2025.108158.
- [6] G. Erceylan, A. Akbarzadeh, and V. Gkioulos, "Leveraging digital twins for advanced threat modeling in cyber-physical systems cybersecurity," *International Journal of Information Security*, vol. 24, art. no. 151, 2025, doi: 10.1007/s10207-025-01043-x.

- [7] P. Empl *et al.*, “A review of digital twins and their application in cybersecurity based on artificial intelligence,” *Artificial Intelligence Review*, vol. 57, art. no. 201, 2024, doi: 10.1007/s10462-024-10805-3.
- [8] S. Suhail, M. Iqbal, K. McLaughlin, B. Lee, and B. Imtiaz, “A framework for enhancing cyber incident response with Security-Enhancing Digital Twins in Cyber-Physical Systems,” *Internet of Things*, vol. 31, art. no. 101547, 2025, doi: 10.1016/j.iot.2025.101547.
- [9] F. Baiardi and V. Sammartino, “Simulation-powered cybersecurity: real-time risk assessment via non-intrusive security twin,” *The Journal of Supercomputing*, vol. 82, art. no. 311, 2026, doi: 10.1007/s11227-026-08454-0.
- [10] M. Repetto, “Cybersecurity Digital Twins: Concept, blueprint, and challenges for multi-ownership digital service chains,” *Journal of Information Security and Applications*, vol. 96, art. no. 104299, 2026, doi: 10.1016/j.jisa.2025.104299.
- [11] S. Moss, “Evaluating the Effectiveness of CSPM Tools in Detecting Cloud Misconfigurations,” preprint, 2026, TechRxiv, doi: 10.36227/techrxiv.177162139.97797546.
- [12] Amazon Web Services, “Security best practices for Amazon S3,” AWS Documentation, accessed Sep. 2026.
- [13] MITRE ATT&CK, “Valid Accounts: Cloud Accounts (T1078.004),” Enterprise ATT&CK knowledge base, accessed Sep. 2026.